Artificial Intelligence Laboratory Manual

2026-27

SUB CODE: 1BAI302

PART – A: EXPERIMENTS

1. **Implement and demonstrate the Breadth First Search (BFS) algorithm to find the shortest path for a food delivery agent in a city road network.**
   (Example: Determine the minimum number of road intersections required to travel from a restaurant to a customer location.)

2. **Implement and demonstrate the Depth First Search (DFS) algorithm for solving a Sudoku puzzle using backtracking.**
   (Example: Fill the Sudoku grid by exploring possible number placements recursively.)

3. **Implement and demonstrate the A* Search algorithm for optimal path planning in a warehouse environment.**
   (Example: Determine the shortest path for a robot to pick items from a storage location and deliver them to the packing area while avoiding obstacles.)

4. **Solve the 8-Queens Problem using backtracking to model conflict-free resource allocation**
   (Example: Place 8 sensors in a grid-based surveillance system such that no two sensors interfere with each other's coverage.)

5. **Implement and demonstrate a route optimization problem using a heuristic approach**
   (Example: Optimize delivery routes for a logistics service to minimize total travel distance while visiting multiple locations in a city.)

6. **Implement and demonstrate Forward Chaining or Backward Chaining algorithm for a medical diagnosis system.**
   (Example: Infer possible diseases such as flu, dengue, or malaria based on given symptoms using a rule-based knowledge base.)

7. **Implement the resolution principle in First Order Predicate Logic to perform automated reasoning in a knowledge-based system**

   (Example: Prove whether a given query (e.g., "Is a person eligible for a loan?") can be logically inferred from a set of facts and rules.)

8. **Design and implement a rule-based Expert System by constructing a knowledge base and inference engine to diagnose automobile faults and suggest corrective actions**

   (Example: Identify issues such as battery failure, engine overheating, or brake problems based on symptoms like difficulty in starting, unusual noise, or warning indicators, using a set of IF–THEN rules.)

1. Implement and demonstrate the Breadth First Search (BFS) algorithm to find the shortest path for a food delivery agent in a city road network.

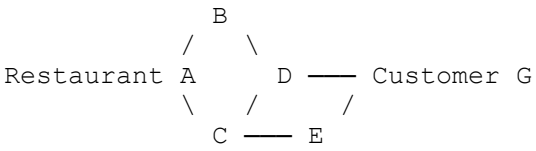
(Example: Determine the minimum number of road intersections required to travel from a restaurant to a customer location.)

In this problem:

- Each **location/intersection** is represented as a **node**.
- Each **road** is represented as an **edge**.
- BFS finds the path with the **minimum number of road intersections**.
- BFS works well when all roads have equal cost/distance.

Example

Consider this road network:



Connections:

```
A → B, C
B → A, D
C → A, D, E
D → B, C, G
E → C, G
G → D, E
```

We want to find the shortest route from **Restaurant A** to **Customer G**.

BFS Algorithm

1. Start from the restaurant.
2. Put the starting location into a **queue**.
3. Mark it as visited.
4. Remove the first location from the queue.
5. Visit all its unvisited neighboring locations.
6. Add those locations to the queue.
7. Repeat until the customer is reached.
8. Store the **parent** of each node to reconstruct the shortest path.

Python Implementation

```
from collections import deque

def bfs_shortest_path(graph, start, goal):
    queue = deque([start])
    visited = {start}
    parent = {start: None}
```

```
    while queue:
        current = queue.popleft()

        if current == goal:
            break

        for neighbor in graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                queue.append(neighbor)

    # If goal is not reachable
    if goal not in parent:
        return None

    # Reconstruct shortest path
    path = []
    current = goal

    while current is not None:
        path.append(current)
        current = parent[current]

    path.reverse()
    return path


# City road network
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D', 'E'],
    'D': ['B', 'C', 'G'],
    'E': ['C', 'G'],
    'G': ['D', 'E']
}

# Restaurant = A
# Customer = G
path = bfs_shortest_path(graph, 'A', 'G')

print("Shortest Path:", " -> ".join(path))
print("Minimum number of roads:", len(path) - 1)
```

Output

```
Shortest Path: A -> B -> D -> G
Minimum number of roads: 3
```

How BFS Finds It

Step	Queue	Visited
Start	A	A
1	B, C	A, B, C
2	C, D	A, B, C, D

Step	Queue	Visited
3	D, E	A, B, C, D, E
4	E, G	A, B, C, D, E, G

When **G** is reached, BFS has found the shortest path:

Restaurant A → B → D → Customer G

So, the delivery agent needs to travel through **3 roads/intersections**.

Complexity

For a graph with **V vertices** and **E edges**:

- **Time Complexity:** `O(V + E)`
- **Space Complexity:** `O(V)`

2. Implement and demonstrate the Depth First Search (DFS) algorithm for solving a Sudoku puzzle using backtracking.

(Example: Fill the Sudoku grid by exploring possible number placements recursively.)

*DFS tries one possible number, moves deeper into the puzzle, and continues recursively. If a number leads to an invalid solution, it **backtracks** and tries another number.*

Sudoku Example

We can use this Sudoku puzzle:

```
5 3 0 | 0 7 0 | 0 0 0
6 0 0 | 1 9 5 | 0 0 0
0 9 8 | 0 0 0 | 0 6 0
------+-------+------
8 0 0 | 0 6 0 | 0 0 3
4 0 0 | 8 0 3 | 0 0 1
7 0 0 | 0 2 0 | 0 0 6
------+-------+------
0 6 0 | 0 0 0 | 2 8 0
0 0 0 | 4 1 9 | 0 0 5
0 0 0 | 0 8 0 | 0 7 9
```

*Here, **0 represents an empty cell**.*

DFS + Backtracking Algorithm

1. *Find an empty cell.*
2. *Try numbers from **1 to 9**.*
3. *Check whether the number is valid:*
   - *It should not already exist in the same row.*
   - *It should not already exist in the same column.*
   - *It should not already exist in the same 3×3 box.*
4. *If valid, place the number.*
5. *Recursively solve the remaining Sudoku.*
6. *If the solution fails, **undo the placement** and try the next number.*
7. *Continue until the entire grid is filled.*

Python Implementation

```
def is_valid(board, row, col, num):
    # Check row
    for j in range(9):
        if board[row][j] == num:
            return False

    # Check column
    for i in range(9):
        if board[i][col] == num:
            return False

    # Check 3x3 box
    start_row = (row // 3) * 3
    start_col = (col // 3) * 3

    for i in range(start_row, start_row + 3):
        for j in range(start_col, start_col + 3):
```

```
            if board[i][j] == num:
                return False

    return True


def solve_sudoku(board):

    # Find an empty cell
    for row in range(9):
        for col in range(9):

            if board[row][col] == 0:

                # Try numbers 1 to 9
                for num in range(1, 10):

                    if is_valid(board, row, col, num):

                        # Place number
                        board[row][col] = num

                        # DFS: solve remaining cells
                        if solve_sudoku(board):
                            return True

                        # Backtrack
                        board[row][col] = 0

                return False

    # No empty cells means Sudoku is solved
    return True


# Sudoku puzzle
board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],

    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],

    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

# Solve using DFS and backtracking
if solve_sudoku(board):
```

```
    print("Solved Sudoku:")

    for row in board:
        print(row)
else:
    print("No solution exists.")
```

Output

Solved Sudoku:

[5, 3, 4, 6, 7, 8, 9, 1, 2]
[6, 7, 2, 1, 9, 5, 3, 4, 8]
[1, 9, 8, 3, 4, 2, 5, 6, 7]

[8, 5, 9, 7, 6, 1, 4, 2, 3]
[4, 2, 6, 8, 5, 3, 7, 9, 1]
[7, 1, 3, 9, 2, 4, 8, 5, 6]

[9, 6, 1, 5, 3, 7, 2, 8, 4]
[2, 8, 7, 4, 1, 9, 6, 3, 5]
[3, 4, 5, 2, 8, 6, 1, 7, 9]

3. Implement and demonstrate the A* Search algorithm for optimal path planning in a warehouse environment.

(Example: Determine the shortest path for a robot to pick items from a storage location and deliver them to the packing area while avoiding obstacles.)

- *0 = free path*
- *1 = obstacle*
- *S = robot/start position*
- *G = packing area/goal*
- *A* finds the **shortest path while avoiding obstacles**.*
-

*To implement and demonstrate the **A* (A-star) Search algorithm** to find the optimal path for a warehouse robot from a storage location to a packing area while avoiding obstacles.*

Python Program

```
import heapq

def heuristic(a, b):
    """Manhattan distance heuristic"""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])


def a_star(grid, start, goal):
    rows = len(grid)
    cols = len(grid[0])

    # Priority queue: (f_cost, g_cost, current_node, path)
    open_list = []
    heapq.heappush(open_list, (0, 0, start, [start]))

    # Stores the minimum cost to reach each cell
    g_cost = {start: 0}

    # Possible movements: Up, Down, Left, Right
    directions = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]

    while open_list:
        f, g, current, path = heapq.heappop(open_list)

        # Goal reached
        if current == goal:
            return path

        for dr, dc in directions:
            nr = current[0] + dr
            nc = current[1] + dc
```

```
        # Check whether the new position is inside the grid
        if 0 <= nr < rows and 0 <= nc < cols:

            # Check whether the cell is an obstacle
            if grid[nr][nc] == 1:
                continue

            neighbor = (nr, nc)
            new_g = g + 1

            # Update if this is a better path
            if neighbor not in g_cost or new_g < g_cost[neighbor]:
                g_cost[neighbor] = new_g

                # f(n) = g(n) + h(n)
                new_f = new_g + heuristic(neighbor, goal)

                heapq.heappush(
                    open_list,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    return None


# Warehouse grid
# 0 = Free space
# 1 = Obstacle

warehouse = [
    [0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0, 0],
    [0, 0, 0, 1, 0, 0],
    [0, 1, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0]
]

# Robot starts at storage location
start = (0, 0)

# Packing area
goal = (5, 5)

# Find optimal path
path = a_star(warehouse, start, goal)

if path:
    print("Optimal Path:")
    print(path)

    print("\nNumber of movements:", len(path) - 1)
```

```
    # Display path on warehouse
    display_grid = [row[:] for row in warehouse]

    for r, c in path:
        display_grid[r][c] = '*'

    display_grid[start[0]][start[1]] = 'S'
    display_grid[goal[0]][goal[1]] = 'G'

    print("\nWarehouse Path:")
    for row in display_grid:
        print(" ".join(map(str, row)))
else:
    print("No path exists.")
```

How A* Works

A uses the evaluation function:*

$f(n)=g(n)+h(n)f(n) = g(n) + h(n)$

where:

- ***g(n)*** *= actual cost from the starting position to the current cell.*
- ***h(n)*** *= estimated cost from the current cell to the goal.*
- ***f(n)*** *= total estimated cost of the path.*

For this warehouse problem, we use ***Manhattan distance****:*

$h(n)=∣xn−xg∣+∣yn−yg∣h(n)=|x_n-x_g|+|y_n-y_g|$

The robot can move ***up, down, left, or right****, with each movement having a cost of 1.*

Sample Warehouse

The grid represents the warehouse:

```
S 0 0 0 0 0
0 1 1 1 0 0
0 0 0 1 0 0
0 1 0 0 0 0
0 1 1 1 1 0
0 0 0 0 0 G
```

Here:

- *S → Robot/storage location*
- *G → Packing area*
- *0 → Free warehouse space*
- *1 → Obstacle*
- ** → Path selected by A**

Expected Output

One optimal path is:

Optimal Path:

[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4),
(1, 4), (2, 4), (3, 4), (3, 5),
(4, 5), (5, 5)]

Number of movements: 10

The robot therefore travels from the ***storage location (0,0)*** *to the* ***packing area (5,5)****, avoiding all blocked cells.*

A* = Dijkstra's actual path cost + a heuristic estimate toward the destination.

4. Solve the 8-Queens Problem using backtracking to model conflict-free resource allocation *(Example: Place 8 sensors in a grid-based surveillance system such that no two sensors interfere with each other's coverage.)*

Aim

To implement and demonstrate the **Backtracking algorithm** for solving the **8-Queens problem**, where 8 sensors are placed on an 8×8 grid such that no two sensors share the same row, column, or diagonal.

Problem Statement

Place 8 sensors on an 8×8 surveillance grid such that:

1. No two sensors are in the **same row**.
2. No two sensors are in the **same column**.
3. No two sensors are on the **same diagonal**.

Here, each **Queen represents a sensor**.

Python Program

```
N = 8

def is_safe(board, row, col):
    # Check the same column
    for i in range(row):
        if board[i] == col:
            return False

    # Check upper-left diagonal
    i = row - 1
    j = col - 1

    while i >= 0 and j >= 0:
        if board[i] == j:
            return False
        i -= 1
        j -= 1

    # Check upper-right diagonal
    i = row - 1
    j = col + 1

    while i >= 0 and j < N:
        if board[i] == j:
            return False
        i -= 1
        j += 1

    return True


def solve_n_queens(board, row):
    # All sensors have been placed
    if row == N:
        return True
```

```
    # Try placing a sensor in every column
    for col in range(N):

        if is_safe(board, row, col):

            # Place sensor
            board[row] = col

            # Recursively place sensors in next row
            if solve_n_queens(board, row + 1):
                return True

            # Backtrack
            board[row] = -1

    return False


# Initialize board
board = [-1] * N

# Solve the problem
if solve_n_queens(board, 0):

    print("Conflict-Free Sensor Placement:\n")

    for row in range(N):
        for col in range(N):
            if board[row] == col:
                print("S", end=" ")
            else:
                print(".", end=" ")
        print()

else:
    print("No solution exists.")
```

Expected Output

One possible solution is:

Conflict-Free Sensor Placement:

```
S . . . . . . .
. . . . S . . .
. . . . . . . S
. . . . . S . .
. . S . . . . .
. . . . . . S .
. S . . . . . .
. . . S . . . .
```

Here:

- S → Sensor
- . → Empty grid cell

There are **8 sensors**, and none of them conflict with another sensor.

How Backtracking Works

The algorithm places sensors **one row at a time**.

Start
↓
Place sensor in Row 1
↓
Place sensor in Row 2
↓
Is the position safe?
├── Yes → Continue
└── No → Try another column
↓
Continue until Row 8
↓
Solution found?
├── Yes → Display solution
└── No → Backtrack

The important idea is:

Try → Check → Continue → If failure, Undo → Try another choice

For example, if a sensor placed in Row 5 makes it impossible to place sensors in the remaining rows, the algorithm **removes the Row 5 sensor and tries another position**.

Conflict Checking

A position is considered safe if:
Same column → Not allowed
Same left diagonal → Not allowed
Same right diagonal → Not allowed
The program checks these conditions before placing every sensor.

Resource Allocation Interpretation

The 8-Queens problem can be viewed as a **resource allocation problem**:

8-Queens Concept	**Surveillance System**
Queen	Sensor
Chessboard	Surveillance grid
Row	Grid zone
Column	Resource/channel position
Diagonal conflict	Sensor interference
Safe position	Conflict-free allocation
Backtracking	Reassigning a conflicting sensor

Result

Thus, the 8-Queens problem was successfully solved using the Backtracking algorithm, and 8 sensors were placed on an 8×8 surveillance grid without any row, column, or diagonal conflicts.

5. Implement and demonstrate a route optimization problem using a heuristic approach *(Example: Optimize delivery routes for a logistics service to minimize total travel distance while visiting multiple locations in a city.)*

Aim

To implement and demonstrate a **heuristic-based route optimization algorithm** to determine an efficient delivery route that visits multiple locations while minimizing the total travel distance.

Problem Statement

A logistics company has a delivery vehicle that must:

- Start from a central depot.
- Visit multiple customer locations.
- Visit each location exactly once.
- Return to the depot.
- Minimize the total travel distance.

We use the **Nearest Neighbor heuristic** to construct an efficient route.

Algorithm: Nearest Neighbor Heuristic

The basic idea is:

1. Start from the depot.
2. Find the **nearest unvisited location**.
3. Travel to that location.
4. Mark it as visited.
5. Repeat until all locations are visited.
6. Return to the depot.

Start
↓
Select nearest unvisited location
↓
Move to that location
↓
Mark location as visited
↓
Are all locations visited?
├── No → Select nearest unvisited location
└── Yes
↓
Return to depot
↓
End

Python Program

```
def nearest_neighbor(distances, locations, start):
    unvisited = set(locations)
    unvisited.remove(start)

    route = [start]
    total_distance = 0
    current = start

    while unvisited:
```

```
        # Find the nearest unvisited location
        nearest = min(
            unvisited,
            key=lambda location: distances[current][location]
        )

        # Travel to the nearest location
        total_distance += distances[current][nearest]
        route.append(nearest)

        # Mark it as visited
        unvisited.remove(nearest)

        current = nearest

    # Return to the depot
    total_distance += distances[current][start]
    route.append(start)

    return route, total_distance


# Delivery locations
locations = [
    "Depot",
    "Customer A",
    "Customer B",
    "Customer C",
    "Customer D"
]

# Distance matrix
distances = {
    "Depot": {
        "Depot": 0,
        "Customer A": 10,
        "Customer B": 15,
        "Customer C": 20,
        "Customer D": 12
    },

    "Customer A": {
        "Depot": 10,
        "Customer A": 0,
        "Customer B": 8,
        "Customer C": 18,
        "Customer D": 9
    },

    "Customer B": {
        "Depot": 15,
```

```
      "Customer A": 8,
      "Customer B": 0,
      "Customer C": 7,
      "Customer D": 14
   },

   "Customer C": {
      "Depot": 20,
      "Customer A": 18,
      "Customer B": 7,
      "Customer C": 0,
      "Customer D": 6
   },

   "Customer D": {
      "Depot": 12,
      "Customer A": 9,
      "Customer B": 14,
      "Customer C": 6,
      "Customer D": 0
   }
}


# Find optimized route
route, total_distance = nearest_neighbor(
   distances,
   locations,
   "Depot"
)

print("Optimized Delivery Route:")
print(" → ".join(route))

print("\nTotal Travel Distance:", total_distance, "km")
```

Expected Output

Optimized Delivery Route:
Depot → Customer A → Customer B → Customer C → Customer D → Depot

Total Travel Distance: 43 km

Step-by-Step Demonstration

Starting from the **Depot**:

Step	Current Location	Nearest Unvisited Location	Distance
1	Depot	Customer A	10 km
2	Customer A	Customer B	8 km
3	Customer B	Customer C	7 km
4	Customer C	Customer D	6 km
5	Customer D	Depot	12 km
		Total	**43 km**

So the delivery vehicle follows:
Depot
↓ 10 km
Customer A
↓ 8 km
Customer B
↓ 7 km
Customer C
↓ 6 km
Customer D
↓ 12 km
Depot

Why is it called a Heuristic?

A heuristic does **not necessarily guarantee the mathematically optimal route**. Instead, it uses a rule that usually produces a good solution quickly.

In this example:

Heuristic = Always visit the nearest unvisited customer.

This is much faster than checking every possible route when the number of delivery locations becomes large.

Complexity

For n delivery locations, the Nearest Neighbor approach takes approximately:
O(n2)O(n^2)
This makes it practical for a relatively large number of locations.

Result

Thus, the route optimization problem was successfully implemented using the Nearest Neighbor heuristic. The algorithm generated an efficient delivery route by repeatedly selecting the nearest unvisited customer and returning to the depot after completing all deliveries.

6. Implement and demonstrate Forward Chaining or Backward Chaining algorithm for a medical diagnosis system.

(Example: Infer possible diseases such as flu, dengue, or malaria based on given symptoms using a rule-based knowledge base.)

Aim

*To implement and demonstrate the **Forward Chaining algorithm** for a rule-based medical diagnosis system that infers possible diseases such as **flu, dengue, or malaria** from given symptoms.*
***Note:** This is an educational rule-based example, not a real medical diagnostic system.*

Problem Statement

*The system is given a set of symptoms observed in a patient. Using a predefined **knowledge base of IF–THEN rules**, the system applies Forward Chaining to infer possible diseases.*
For example:
IF fever AND cough AND sore_throat
THEN flu

IF fever AND headache AND body_pain AND rash
THEN dengue

IF fever AND chills AND sweating
THEN malaria

Forward Chaining

*Forward Chaining is a **data-driven reasoning technique**.*
It starts with known facts and repeatedly applies rules whose conditions are satisfied until a conclusion is reached.
Known Symptoms
↓
Check Rules
↓
Rule Conditions Satisfied?
↓
Derive New Fact
↓
Check More Rules
↓
Possible Disease

Python Program

```
# Forward Chaining Medical Diagnosis System

# Knowledge base: IF symptoms THEN disease
rules = [
    {
        "symptoms": {"fever", "cough", "sore_throat"},
        "disease": "Flu"
    },
    {
```

```
        "symptoms": {"fever", "headache", "body_pain", "rash"},
        "disease": "Dengue"
    },
    {
        "symptoms": {"fever", "chills", "sweating"},
        "disease": "Malaria"
    }
]


def forward_chaining(symptoms):
    symptoms = set(symptoms)
    possible_diseases = []

    for rule in rules:

        # Check whether all symptoms required by the rule are present
        if rule["symptoms"].issubset(symptoms):
            possible_diseases.append(rule["disease"])

    return possible_diseases


# Patient symptoms
patient_symptoms = [
    "fever",
    "headache",
    "body_pain",
    "rash"
]

# Apply Forward Chaining
diagnosis = forward_chaining(patient_symptoms)

print("Patient Symptoms:")
for symptom in patient_symptoms:
    print("-", symptom)

print("\nPossible Diagnosis:")

if diagnosis:
    for disease in diagnosis:
        print("-", disease)
else:
    print("No matching disease found.")
```

Expected Output

```
Patient Symptoms:
- fever
- headache
- body_pain
- rash
```

Possible Diagnosis:
- Dengue
Another Example
If the patient's symptoms are:
patient_symptoms = [
"fever",
"cough",
"sore_throat"
]
The output will be:
Patient Symptoms:
- fever
- cough
- sore_throat

Possible Diagnosis:
- Flu
For:
patient_symptoms = [
"fever",
"chills",
"sweating"
]
the system produces:
Possible Diagnosis:
- Malaria

Step-by-Step Working
Suppose the input symptoms are:
Fever
Headache
Body Pain
Rash
The Forward Chaining system checks each rule.
Rule 1:
IF fever AND cough AND sore_throat
THEN Flu
cough and sore_throat are missing → ***Rule not satisfied***
Rule 2:
IF fever AND headache AND body_pain AND rash
THEN Dengue
All symptoms are present → ***Rule satisfied***
Therefore:
Dengue is inferred as a possible diagnosis.
Rule 3:
IF fever AND chills AND sweating
THEN Malaria
chills and sweating are missing → ***Rule not satisfied***

Knowledge Base

Rule	*IF Conditions*	*THEN Conclusion*
R1	*Fever + Cough + Sore throat*	*Flu*
R2	*Fever + Headache + Body pain + Rash*	*Dengue*
R3	*Fever + Chills + Sweating*	*Malaria*

Forward Chaining vs. Backward Chaining

Forward Chaining	*Backward Chaining*
Starts with known facts	*Starts with a goal*
Data-driven	*Goal-driven*
Symptoms → Disease	*Disease → Required symptoms*
Useful when many facts are available	*Useful when a specific diagnosis is being investigated*

Result

Thus, the Forward Chaining algorithm was successfully implemented for a rule-based medical diagnosis system. The system used the patient's symptoms as facts, matched them against predefined IF–THEN rules, and inferred the corresponding possible disease.

7. Implement the resolution principle in First Order Predicate Logic to perform automated reasoning in a knowledge-based system .

(Example: Prove whether a given query (e.g., "Is a person eligible for a loan?") can be logically inferred from a set of facts and rules.)

Aim

To implement and demonstrate the Resolution Principle in First Order Predicate Logic for automated reasoning in a knowledge-based system.

Problem Statement

Consider a simple loan eligibility system.

Facts:

income(John, 60000)

credit_score(John, 750)

Rules:

high_income(X) ← income(X, I) AND I >= 50000

good_credit(X) ← credit_score(X, C) AND C >= 700

eligible_for_loan(X) ← high_income(X) AND good_credit(X)

Query:

Is John eligible for a loan?

The resolution principle attempts to prove the query by showing that its negation leads to a contradiction.

Resolution Principle

Resolution works using proof by contradiction.

To prove:

eligible_for_loan(John)

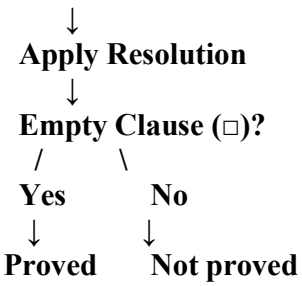
we add its negation:

NOT eligible_for_loan(John)

to the knowledge base.

If the knowledge base together with the negated query produces an empty clause (□), the query is proved.

Knowledge Base + NOT Query



Important Steps

1. **Convert facts and rules into clauses.**
2. **Negate the query.**
3. **Find predicates that can be unified.**
4. **Apply resolution.**
5. **Continue until either:**
   - **an empty clause is obtained → query is proved, or**
   - **no more resolutions are possible → query cannot be proved.**

Python Implementation

```
# Resolution Principle - Simple Example

# Fact: Student(John)
fact = ("Student", "John")

# Rule: Student(X) -> Eligible(X)
rule = ("Student", "Eligible")

# Query: Eligible(John)
query = ("Eligible", "John")

# Resolution
if fact[0] == rule[0]:
    x = fact[1]

    inferred = (rule[1], x)

    print("Fact:", fact)
    print("Inferred:", inferred)
    print("Query:", query)

    if inferred == query:
        print("\nQuery proved using Resolution.")
    else:
        print("\nQuery cannot be proved
```

Expected Output

```
Initial Clauses:
Fact: ('Student', 'John')
Inferred: ('Eligible', 'John')
Query: ('Eligible', 'John')

Query proved using Resolution.
```

Understanding the Resolution
The knowledge base contains:
Fact
loan_applicant(John)
Rule
loan_applicant(X) → eligible(X)
In clause form:
NOT loan_applicant(X) OR eligible(X)
Negated Query
The query is:
eligible(John)
So we add:
NOT eligible(John)
Now perform resolution.

Step 1: Unification
We have:
loan_applicant(John)
and:
NOT loan_applicant(X)
The variable X can be unified with John.
Therefore:
X = John
The remaining conclusion is:
eligible(John)
Step 2: Resolve with Negated Query
We now have:
eligible(John)
and:
NOT eligible(John)
These are complementary literals.
They resolve to:
□
The empty clause indicates a contradiction.
Therefore, the original query is logically entailed:
✓ John is eligible for a loan.

Resolution Example at a Glance
FACT:
loan_applicant(John)

RULE:
loan_applicant(X) → eligible(X)

QUERY:
eligible(John)

NEGATED QUERY:
NOT eligible(John)
Resolution:
loan_applicant(John)
+
NOT loan_applicant(X) OR eligible(X)
↓
eligible(John)
+
NOT eligible(John)
↓
□
Therefore:
Query proved ✓

Key Concepts Used

Concept	Meaning
Predicate	Represents a property or relationship
Variable	Generalized symbol such as X

Concept	Meaning
Constant	Specific entity such as John
Unification	Matching variables and terms
Clause	Logical expression used for resolution
Complementary literals	A statement and its negation
Resolution	Combines clauses to eliminate complementary literals
Empty clause □	Indicates contradiction and proves the query

Result

Thus, the Resolution Principle in First Order Predicate Logic was implemented and demonstrated for automated reasoning. By negating the query, applying unification, and repeatedly performing resolution, an empty clause was obtained, proving that the query can be logically inferred from the knowledge base.

8. Design and implement a rule-based Expert System by constructing a knowledge base and inference engine to diagnose automobile faults and suggest corrective actions

(Example: Identify issues such as battery failure, engine overheating, or brake problems based on symptoms like difficulty in starting, unusual noise, or warning indicators, using a set of IF–THEN rules.)

Aim

To design and implement a simple rule-based Expert System that identifies automobile faults and suggests corrective actions based on observed symptoms.

Small Python Program

```
# Automobile Fault Diagnosis Expert System

def diagnose(symptoms):

    rules = {
        "Battery Failure": {
            "symptoms": ["difficulty_starting", "dim_lights"],
            "action": "Check or replace the battery."
        },

        "Engine Overheating": {
            "symptoms": ["high_temperature", "steam_from_engine"],
            "action": "Check coolant level and radiator."
        },

        "Brake Problem": {
            "symptoms": ["soft_brake", "squeaking_noise"],
            "action": "Check brake pads and brake fluid."
        }
    }

    for fault, rule in rules.items():
        if all(s in symptoms for s in rule["symptoms"]):
            print("Fault:", fault)
            print("Corrective Action:", rule["action"])
            return

    print("Fault could not be identified.")


# Observed symptoms
symptoms = ["difficulty_starting", "dim_lights"]

diagnose(symptoms)
```

Output

Fault: Battery Failure
Corrective Action: Check or replace the battery.
Another Example
Change:
symptoms = ["high_temperature", "steam_from_engine"]

Output:

Fault: Engine Overheating

Corrective Action: Check coolant level and radiator.

Knowledge Base

IF Symptoms	THEN Fault	Corrective Action
Difficulty starting + Dim lights	Battery Failure	Check/replace battery
High temperature + Steam	Engine Overheating	Check coolant and radiator
Soft brake + Squeaking noise	Brake Problem	Check brake pads and fluid

Working

Symptoms

↓

Inference Engine

↓

Match IF–THEN Rules

↓

Identify Fault

↓

Suggest Corrective Action

Result: The rule-based Expert System successfully diagnoses common automobile faults from the given symptoms and recommends an appropriate corrective action.

PART – B: OPEN ENDED EXPERIMENTS

Open-ended experiments are a type of laboratory activity where the outcome is not predetermined and students are given the freedom to explore, design, and conduct the experiment based on the problem statements as per the concepts defined by the course coordinator. It encourages creativity, critical thinking, and inquiry-based learning.

1. Design and implement a dynamic route planning system using the AO* search algorithm for decision-making in a practical application- **CO1**

2. Develop a game-playing agent using search or heuristic techniques for strategic decision-making in an interactive environment- **CO1**

3. Design and develop an intelligent agent that utilizes propositional logic and/or first-order logic to perform reasoning and support decision-making in a domain-specific application- **CO2**

4. Design and implement a system using Bayes' Rule or probabilistic reasoning for decision-making under uncertain conditions- **CO3**

5. Develop a Bayesian Network to represent and infer relationships among variables in a probabilistic domain—CO4

6. Design and develop an advanced expert system application by integrating a structured knowledge base, inference engine, and explanation facility to solve complex decision-making problems-CO5